Find vulnerabilities in assets

I am currently working on an assignment which asks me to find vulnerability in wild. I want to you to help me find vulnerability in this assets. I dont have any experience to find vulnerablity so you have to teach me how to do that. The following areas and paths under domain https://www.simscale.com:

User Account & Security Includes all aspects of user accounts, including signup, login, password management, API keys, and session handling.

Paths: /signup/*, /signin/*, /dashboard/profile, /dashboard/security, /dashboard/api_keys

User registration and following onboarding Login Password reset Change password (login required) API key management (login required) Dashboard Central space for project and permission management.

Paths: /dashboard/*

Dashboard (login required). Workbench Covers the complete simulation workflow, including CAD import and editing, meshing, simulation setup and execution, and post-processing.

Paths: /workbench/*

Workbench (login required), opens a public project in view-only mode. Public project library Paths: /projects/*

Public projects Detail page of a public project They are allowed to test becasue I find it in intigriti.com Here is the scope In scope Focus areas We'd like you to test our core product: The platform with it's web-based user interfaces and the APIs which are listed in the assets above.

Everything around the user account, from registration, login (both via password or SSO), password change, API keys and in general the security of our web session handling. Management of projects, folders, spaces and the sharing settings. CAD upload and import. CAD editing. Simulation and mesh setup. The platform supports more than 15 simulation types, each contains hundreds of settings. This includes CSV upload, Formulas, and custom materials. Table and plot result download. The integrated graphical post-processor. Beside the classic web application vulnerabilities (XSS, SQLi, RCE, etc.), we are especially interested in vulnerabilities that affect our customer's trust into our platform, such as:

Access to private customer data (e.g. projects, CAD files, simulation results) Privilege escalation (both vertical and horizontal) Account takeover Getting started To get an impression of the functionality of the workbench we recommend to walk through one or more tutorials which are also available as video:

Fluid Flow, 15 minutes video Structural Analysis, 15 minutes video View changes Out of scope Highlighted Intercom chat and support Please don't interact with the Intercom chat and don't contact our support via email or phone. It's meant for user and customer support.

Public user profiles and projects The names, descriptions, creation date, last modification date etc. of users and public projects are meant to be public, they are not considered to be sensitive. If information is not mentioned in the web application, it is not sensitive by default. Any information disclosure that does not contain PII or other information that cannot directly be used to cause harm to the company or its users, on endpoints mentioned below will be closed as out of scope. This applies to

https://www.simscale.com/api/v1/projects/{username} https://www.simscale.com/api/v1/projects/{username}/{projectname} https://www.simscale.com/api/v1/users/{username} https://www.simscale.com/wp-json/wp/v2/users https://www.simscale.com/forum/users/{username}.json and other endpoints that disclose similar information as the above. Third-party services and API key disclosure SimScale integrates third-party services. This includes

Intercom chat Hubspot forms New Relic monitoring Geolocation APIs Google Maps and other services listed at https://www.simscale.com/data-privacy/. Vulnerabilities in those services are out of scope.

Leaked/disclosed API keys for those services are out of scope.

Website (WordPress) The SimScale website is based on the Open-Source software WordPress. Before creating a submission about the website please consider to validate your findings against the WordPress platform at https://wordpress.com/wordpress-free/ and to report it to their bug bounty program.

In general no bug bounty is payed for findings in the website, but we might consider to pay a bounty for severe findings.

WordPress username enumeration and disclosure (i.e. https://www.simscale.com/wp-json/wp/v2/users) is out of scope.

Broken Links in articles and posts

Forum (Discourse) The SimScale forum is based on the Open-Source software Discourse. Before creating a submission about the forum please consider to also validate your findings against the Discourse demo platform at https://try.discourse.org/ and to report it to their bug bounty program.

In general no bug bounty is payed for findings in the forum, but we might consider to pay a bounty for severe findings.

Forum user information disclosure (i.e. https://www.simscale.com/forum/users/{username}.json) is out of scope.

Application API key disclosure without proven business impact Pre-Auth Account takeover/OAuth squatting Self-XSS that cannot be used to exploit other users CORS misconfiguration on non-sensitive endpoints Missing cookie flags Missing security headers Cross-site Request Forgery with no or low impact Presence of autocomplete attribute on web forms Reverse tabnabbing By-passing signup limits (plus email etc.) Bypassing rate-limits or the non-existence of rate-limits Best practices violations (password complexity, expiration, re-use, etc.) Clickjacking without proven impact/unrealistic user interaction Sessions not being invalidated (logout, enabling 2FA, etc.) Tokens leaked to third parties Anything related to email spoofing, SPF, DMARC or DKIM Content injection without being able to modify the HTML Username/email enumeration Email bombing HTTP Request smuggling without any proven impact Homograph attacks XMLRPC enabled Banner grabbing/Version disclosure Same-site scripting Subdomain takeover without taking over the subdomain Arbitrary file upload without proof of the existence of the uploaded file Blind SSRF without proven business impact (pingbacks are not sufficient) Host header injection without proven business impact General In case that a reported vulnerability was already known to the company from their own tests, it will be flagged as a duplicate Theoretical security issues with no realistic exploit scenario(s) or attack surfaces, or issues that would require complex end user interactions to be exploited Spam, social engineering and physical intrusion DoS/DDoS attacks or brute force attacks Vulnerabilities that only work on software that no longer receive security updates Attacks requiring physical access to a victim's computer/device, man in the middle or compromised user accounts Recently discovered zero-day vulnerabilities found in in-scope assets within 14 days after the public release of a patch or mitigation may be reported, but are usually not eligible for a bounty Reports that state that software is out of date/vulnerable without a proof-of-concept Out of scope domains Any domain that is not listed in the Domains section, is out of scope for this program

2 previous messages

I'll treat this as authorized bug-bounty learning, and keep it inside the program rules you pasted. First thing I'm going to do is sanity-check the public program scope, because bounty scopes can change and testing the wrong area is the easiest way to get burned.

Searched web 1 time Searched web 1 time

- Searched web for `Intigriti SimScale bug bounty scope simscale.com`

Yes, I can help you learn this safely and in-scope.

I checked the public Intigriti listing, and the key rules still match what you pasted: use an `@intigriti.me` address, avoid Intercom/support, avoid excessive resource use, and automated

tooling is limited to **max 2 requests/second**. Source: SimScale Intigriti program.

Best Beginner Strategy Focus on bugs that are valuable and realistic:

1. **Broken access control / IDOR** This is likely the best area for SimScale. Create two accounts:

    a. Account A: owns a project, folder, API key, uploaded CAD file
    b. Account B: should not have access

Then test whether B can access A's resources by changing IDs in requests:

    c. project IDs
    d. folder IDs
    e. space/team IDs
    f. simulation IDs
    g. mesh/result/download IDs
    h. API key IDs

You are looking for: "Account B can view, edit, clone, download, delete, or run something owned by Account A."

2. **Sharing and permission bugs** Test project sharing carefully:

    a. viewer can edit?
    b. viewer can download private result files?
    c. removed user still has access?
    d. public/private toggle leaks private files?
    e. old download links still work after permissions change?
    f. invited user can escalate role by changing request body, e.g. `viewer` to `owner`

3. **API key authorization** In `/dashboard/api_keys`, generate an API key and compare:

    a. browser session access
    b. API key access
    c. revoked API key access
    d. API key from Account A used against Account B resources

Good finding: revoked or low-privilege API key can still access private projects/results.

4. **File upload / CAD import** Test with harmless files only. Look for:

    a. uploaded file visible to another user
    b. private CAD file downloadable from public URL

c. filename XSS in project/workbench UI
d. unsafe file preview behavior
e. path traversal-style filenames like `../test.step` changing displayed/downloaded names

Avoid heavy CAD files or anything that consumes lots of compute.

5. **Stored XSS with real impact** Try harmless payloads in places that other users can view:

a. project name
b. project description
c. folder name
d. material name
e. simulation name
f. CSV column labels
g. plot/table labels
h. CAD part names if editable

Example harmless test payload:

```
<img src=x onerror=alert(document.domain)>
```

Self-XSS is out of scope, so the important test is: can Account A store it somewhere and Account B trigger it?

How To Test Practically Use Burp Suite Community or OWASP ZAP.

Workflow:

1. Log in as Account A.
2. Create a project, upload a small file, create folders/spaces/shares.
3. Capture requests while opening, editing, downloading, sharing.
4. Copy interesting request URLs and bodies.
5. Log in as Account B in another browser/profile.
6. Replay Account A's request using Account B's cookies/session.
7. Record whether the server blocks it.

A valid access-control bug usually looks like:

```
Expected: Account B receives 403/404.
Actual: Account B receives Account A's private project/result/file data.
```

High-Value Test Cases For This Program Start here:

- Can B open `/workbench/*` project data for A's private project by changing project ID?
- Can B download A's CAD/result/table/plot files from a copied URL?
- Can a public project expose private simulation result files?
- Can a viewer modify simulation settings by replaying edit requests?
- Can a removed collaborator still access the project through cached/API endpoints?
- Can an API key access more than the owning user can access?
- Can revoked API keys still work?
- Can project role be changed by editing JSON like `"role":"viewer"` to `"role":"owner"`?
- Can private project metadata leak through `/api/v1/projects/*` beyond what is public?

What Not To Spend Time On These are listed as out of scope or usually low value:

- missing cookie flags
- missing security headers
- username/email enumeration
- rate-limit bypass
- WordPress user enumeration
- forum/Discourse user disclosure
- third-party API keys
- self-XSS
- brute force
- DoS or heavy compute testing

Report Template When you find something, document it like this:

```
Title:
Private project result files accessible by unauthorized user

Impact:
An attacker with a normal account can download private CAD/simulation result
data from another user's project.

Steps:
1. Account A creates private project.
2. Account A uploads CAD file / runs simulation / exports result.
3. Account A captures download request: ...
4. Account B logs in with no access to the project.
5. Account B sends the same request with their own session.
```

6. Server returns the private file.

Expected:
Account B should receive 403 or 404.

Actual:
Account B receives the private file.

Evidence:
Screenshots, request/response snippets, affected IDs, timestamps.

Scope:
https://www.simscale.com/workbench/*

My recommendation: start with **two-account authorization testing** around project sharing, workbench downloads, and API keys. That is the cleanest path to a real, bounty-worthy vulnerability here.

I want to find Broken access control / IDOR For now, I have created two account, oppened burp suite, here is the Now I am in this step In the browser window of account A, open a project you created. Switch to Burp Suite $\rightarrow$ Proxy $\rightarrow$ HTTP history. Find the GET /api/v1/projects/... request in the list. Click on the request $\rightarrow$ view the Response (response packet) on the right side. Record these two values (very important): CanonicalProjectOwner: (e.g., victim_user) $\rightarrow$ This is XXXX CanonicalPublicProjectName: (e.g., secret_project_123) $\rightarrow$ This is YYYY But I found the responses are 一个是 404 HTTP/2 404 Not Found Content-Type: application/json Date: Tue, 05 May 2026 05:33:51 GMT X-Content-Type-Options: nosniff Cache-Control: no-cache, private Trace-Id: 1ee93b29 Span-Id: jlfn555t Vary: Authorization X-Frame-Options: SAMEORIGIN Strict-Transport-Security: max-age=31536000; includeSubDomains; preload X-Cache: Error from cloudfront Via: 1.1 0b8c49517c533bb6e0c14033e0c899b0.cloudfront.net (CloudFront) X-Amz-Cf-Pop: SYD62-P3 Alt-Svc: h3=":443"; ma=86400 X-Amz-Cf-Id: pT8A4MFwtAaEESEsopXLk_94X_zPw4kDNMmxT8SVXGjNer-DSguk7Q== X-Xss-Protection: 1; mode=block Referrer-Policy: strict-origin-when-cross-origin Vary: Origin

{ "spanId" : "jlfn555t", "traceId" : "1ee93b29", "status" : 404, "timestamp" : 1777959231230, "isoTimestamp" : "2026-05-05T05:33:51.230891118Z", "message" : "No forum topic exists for this project", "errorCode" : "not-found", "items" : [{ "message" : "No forum topic exists for this project", "errorCode" : "not-found" }] } 一个是 200 HTTP/2 200 OK Content-Type: application/json Content-Length: 3 Date: Tue, 05 May 2026 05:33:51 GMT Cache-Control: no-cache, private Trace-Id: 771163ad

Span-Id: fngrw42r Vary: Authorization X-Frame-Options: SAMEORIGIN Strict-Transport-Security: max-age=31536000; includeSubDomains; preload Referrer-Policy: strict-origin-when-cross-origin Cross-Origin-Opener-Policy: same-origin-allow-popups X-Content-Type-Options: nosniff X-Cache: Miss from cloudfront Via: 1.1 0b8c49517c533bb6e0c14033e0c899b0.cloudfront.net (CloudFront) X-Amz-Cf-Pop: SYD62-P3 Alt-Svc: h3=":443"; ma=86400 X-Amz-Cf-Id: 7ndJZGDDKUoAmFUxd9x0Dn1yzmjzuo7cnOsUzQrJMS1VRap8Af7nSA== X-Xss-Protection: 1; mode=block Vary: Origin

[ ] 还有一个是 api/v1/permission/projets/victimstudent HTTP/2 200 OK Content-Type: application/json Date: Tue, 05 May 2026 05:33:51 GMT X-Content-Type-Options: nosniff Cache-Control: no-cache, private Trace-Id: 7bfbf4c3 Span-Id: izxf43zu X-Frame-Options: SAMEORIGIN Strict-Transport-Security: max-age=31536000; includeSubDomains; preload Referrer-Policy: strict-origin-when-cross-origin Cross-Origin-Opener-Policy: same-origin-allow-popups X-Cache: Miss from cloudfront Via: 1.1 0b8c49517c533bb6e0c14033e0c899b0.cloudfront.net (CloudFront) X-Amz-Cf-Pop: SYD62-P3 Alt-Svc: h3=":443"; ma=86400 X-Amz-Cf-Id: hGNIWk6FCkD9q5NoHPzN_RkTd7V0YqbPMl3f1dQw9ZSLX9PQlxFnfQ== X-Xss-Protection: 1; mode=block Vary: Origin

{ "permissions" : [{ "scope" : { "scopeType" : "USER", "uidExt" : 541445758, "username" : "VictimStudent", "owner" : true }, "permission" : "FULL_ACCESS" }, { "scope" : { "scopeType" : "GROUP", "groupName" : "PUBLIC", "owner" : false }, "permission" : "COPY" }] }

You're looking at the wrong-but-useful requests. Nothing is broken yet.

What those responses mean:

- The `404` says only: "this project has no forum topic." Ignore it for IDOR.
- The `200 []` means an endpoint returned an empty list. Also probably not the project identity endpoint you need.
- The `api/v1/permission/projects/victimstudent` response is useful: it tells you the project/user permission state. It shows:
  - `VictimStudent` has `FULL_ACCESS`
  - `PUBLIC` has `COPY`

That likely means the project is currently public-copyable or at least exposed in some public way, so it may not be good for private-data IDOR testing yet.

Next Step In Account A, make sure the project is private first.

Then in Burp HTTP history, filter/search for:

`/api/v1/projects/`

You want a request shaped more like one of these:

```
GET /api/v1/projects/VictimStudent/project-name
GET /api/v1/projects/VictimStudent/project-name/...
```

The useful response should contain project details such as:

```
"canonicalProjectOwner": "VictimStudent"
"canonicalPublicProjectName": "some-project-name"
```

or similar casing like:

```
"canonicalProjectOwner"
"canonicalPublicProjectName"
```

If you do not see it, try this:

1. In Account A, open the project detail page from `/projects/*`, not only the workbench.
2. Refresh the page with Burp intercept off.
3. In Burp HTTP history, search for the project name or username: `VictimStudent`.
4. Click JSON responses with `200 OK`.
5. Look for fields containing `canonical`, `projectId`, `uid`, `permission`, `owner`, `visibility`, or `publicProjectName`.

Important Test Once you find a real project-detail request, send the exact same request using Account B's session.

Expected result if private:

```
403 Forbidden or 404 Not Found
```

Potential vulnerability:

```
200 OK with Account A's private project data
```

For now, do not report anything from the `permission/projects/victimstudent` response alone. It looks like normal permission metadata, and if `PUBLIC: COPY` is intentional, it may be expected for a public/copyable project.

I changed the "canonicalProjectOwner" and "canonicalPublicProjectName" in account B which is the attacker to account A's information. Did this in repeater. and send. Here is the response HTTP/2 200 OK Content-Type: application/json Content-Length: 1595 Date: Tue, 05 May 2026 05:54:07 GMT Cross-Origin-Opener-Policy: same-origin-allow-popups Trace-Id: i58QyCvz Span-Id: waxbmq3h Cache-Control: private Expires: Thu, 01 Jan 1970 00:00:00 UTC Strict-Transport-Security: max-age=31536000; includeSubDomains; preload Vary: Authorization X-Content-Type-Options: nosniff X-Frame-Options: DENY X-Xss-Protection: 0 Referrer-Policy: strict-origin-when-cross-origin X-Cache: Miss from cloudfront Via: 1.1 0b8c49517c533bb6e0c14033e0c899b0.cloudfront.net (CloudFront) X-Amz-Cf-Pop: SYD62-P3 Alt-Svc: h3=":443"; ma=86400 X-Amz-Cf-Id: I9-dkCADm3Eq56a8sjhSikoDoXHz-kpBk1RgBlhCofKwsRLV_1tHhQ== Vary: Origin

{ "CanonicalProjectOwner" : "VictimStudent", "CanonicalPublicProjectName" : "test1_2091111164", "categories" : ["ARCHITECTURE_CONSTRUCTION"], "creationDate" : "May 02, 2026, 11:31:40 AM", "description" : "test1", "isShared" : false, "isSharedWithRequester" : false, "isSharedWithSupport" : null, "lastModificationDate" : "May 02, 2026, 11:31:40 AM", "measurementSystem" : "SI", "noIndex" : false, "numberOfCopies" : 0, "numberOfGeometries" : 0, "numberOfMeshes" : 0, "numberOfSimulationRuns" : 0, "numberOfSimulations" : 0, "numberOfViews" : 0, "ownerName" : "VictimStudent", "parentProject" : null, "projectIdExt" : "3155508863917706347", "projectName" : "Test1", "publicPermission" : { "readPreview" : true, "read" : true, "write" : false, "billableAction" : false, "getCopy" : true, "manage" : false, "unlisted" : false }, "publicProjectName" : "test1_2091111164", "requesterPermissions" : { "canPreviewProject" : true, "canReadProject" : true, "canCopyProject" : true, "canWriteProject" : false, "canExecuteProjectBillableAction" : false, "canManageProject" : false, "canMoveProjectToPersonalSpace" : false, "canListProjectPermissions" : false, "canEditProjectPermissions" : false, "canShareProjectWithSpaceMembers" : false, "canShareProjectWithOrganizationMembers" : false, "canShareProjectWithAnyone" : false, "canMakeProjectPublic" : false, "canDeleteProject" : false }, "shareWithSupportExpiresAt" : null, "tags" : [ ], "thumbnailUrl" : null }